



OAuth2 Grant Types

- To get an access token, the client application obtains authorization from the resource owner.
- The authorization is expressed in the form of an authorization grant.
- Client Application uses Authorization grant given by Resource Owner to get an access token from Authorization Server

OAuth2 defines 4 Authorization grant types:

- 1) Authorization_code
- 2) Implicit
- 3) Resource owner password credentials
- 4) Client credentials

***Refresh Token - Can also be called as Grant Type.**

***Refresh Token can be used to get the Access Token.**

Q) What is the Use of Grant Type?

Ans: Grant Type is used to get the Access Token.

Q) Who needs the Grant Type?

Ans: Client Application

Q) Who gives the Grant Type?

Ans: Resource Owner

Q) Who will provide Access Token?

Ans: Authorization Server.

Q) What is the use of Access Token?

Ans: Access Token required to access the Resources or Resource Endpoints

Q) Who will validate Access Token?

Ans: Resource Server

Q) Who will protect the Resources Access Token?

Ans: Resource Server

Authorization Code Grant Type

- The Authorization Code Grant Type is the most commonly used OAuth 2.0 grant type among OAuth2 Grant Types.
- It is used by both web apps and mobile apps to get an access token after a user authorizes an app.
- At a high level, the flow has the following steps:
 - ✓ The application opens a browser to send the user to the OAuth server
 - ✓ The user sees the authorization prompt and approves the app's request
 - ✓ The user is redirected back to the application with an authorization code in the query string
 - ✓ The application gets access token by using authorization code
- The Authorization Code flow is best used in web and mobile apps. Since the Authorization Code grant has the extra step of exchanging the authorization code for the access token, it provides an additional layer of security.

Working Steps:

- 1) Take Workspace : **OAuth2-Labs**
- 2) Copy any existing Maven Project - BookPriceMS (Simple One) into **OAuth2-Labs** Workspace
- 3) Import as Maven Project.
- 4) Rename the Project - **authorization-code-type-server**
- 5) Rename the package as **com.coursecube.oauth2**
- 6) Update pom.xml
- 7) Update **application.properties**

```
spring.application.name= authorization-code-type-server
```

```
server.port=12345
```

```
security.user.name=srinivas
```

```
security.user.password=srinivas
```

- 8) Update Boot Main Application : **MyApplication.java**
- 9) Update **BookPriceController.java**
- 10) Update **BookPriceInfo.java**
- 11) Write the **ResourceServer**
- 12) Write the **AuthorizationServer**
- 13) Start the Boot Application

Running Steps:

Step1: Send the Request to Authorizarion Server with the following URL to Get the User's Permission

- A) Eendpoint - /oauth/authorize
- B) Query Parameters
 - client_id,
 - redirect_uri,
 - response_type,
 - scope

http://localhost:12345/oauth/authorize?client_id=jlcapp&redirect_uri=http://localhost:5000/callback&response_type=code&scope=read_profile

- Above used Query parameters are described below

response_type=code	This tells the authorization server that the application is initiating the authorization code flow
client_id	The public identifier for the application, obtained when the developer first registered the application
redirect_uri	Tells the authorization server where to send the user back to after they approve the request
scope	One or more space-separated strings indicating which permissions the application is requesting

- If the user approves the request, the authorization server will redirect the browser back to the redirect_uri specified by the application, adding a code to the query string.
- This code randomly generated by the Authorization Server which must be used by the client to request for an access token

<http://localhost:5000/callback?code=N7hSH3>

Step 2: Send the POST Request to Authorizariion Server with the following to get an access token using the previously received Authorization Code

A) Eendpoint - /oauth/ token

B) Query Parameters

- clientId,
- clientSecret
- code received
- redirect_uri,
- grant_type,
- scope

```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H
"content-type: application/x-www-form-urlencoded" -d
"code=N7hSH3&grant_type=authorization_code&redirect_uri=http://localho
st:5000/callback&scope=read_profile"
```

The application makes a POST request to the service's token endpoint with the following parameters:

grant_type=authorization_code	This tells the token endpoint that the application is using the Authorization Code grant type
code	The application includes the authorization code it was given in the redirect
redirect_uri	The same redirect URI that was used when requesting the code
scope	One or more space-separated strings indicating which permissions the application is requesting
client_id	The application's client ID.
client_secret	The application's client secret. This ensures that the request to get the access token is made only from the application, and not from a potential attacker that may have intercepted the authorization code

The token endpoint will verify all the parameters in the request, ensuring the code hasn't expired and that the client ID and secret match. If everything is ok, it will generate an access token and return it in the response!



```
{  
"access_token":"9691a988-476d-4f04-8bdc-32472947821e",  
"token_type":"bearer",  
expires_in:119,  
"scope":"read_profile"  
}
```

Step 3: Now access the BookPrice by providing the retrieved access

You can now Invoke your Resources or REST API with the Access Token received above.

curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization: Bearer 9691a988-476d-4f04-8bdc-32472947821e"

authorization-code-type-server – Files Required

1. MyApplication.java	2. BookPriceInfo.java
3. BookPriceController.java	4. MyResourceServer.java
5. MyAuthorizationServer.java	6. application.properties
7. pom.xml	

1. MyApplication.java

```
package com.coursecube.oauth2;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
/*  
 * @Author : Srinivas Dande  
 * @company : CourseCube  
 * @website : www.coursecube.com  
 */  
@SpringBootApplication  
public class MyApplication {  
    public static void main(String args[]) {  
        SpringApplication.run(MyApplication.class, args);  
    }  
}
```



2. BookPriceInfo.java

```
package com.coursecube.oauth2;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;

    //Constrcutors
    //Setters and Getters
}
```

3. BookPriceController.java

```
package com.coursecube.oauth2;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@RestController
@RequestMapping("/myapi")
public class BookPriceController {

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId) {
        System.out.println("---BookPriceController---getBookPrice()----: ");
        BookPriceInfo bookPriceInfo = new BookPriceInfo(bookId, 2500, 20);
        return bookPriceInfo;
    }
}
```



4. MyResourceServer.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Configuration
@EnableResourceServer
public class MyResourceServer extends ResourceServerConfigurerAdapter{

    @Override
    public void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests()
            .anyRequest()
            .authenticated()
            .and()
            .requestMatchers()
            .antMatchers("/myapi/**");
    }
}
```

5. MyAuthorizationServer.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.config.annotation.configurers.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */

@Configuration
@EnableAuthorizationServer
public class MyAuthorizationServer extends AuthorizationServerConfigurerAdapter {
```



```
@Override
public void configure(ClientDetailsServiceConfigurer clients) throws Exception {

    clients.inMemory()
        .withClient("jlcapp")
        .secret("jlc123")
        .redirectUri("http://localhost:5000/callback")
        .authorizedGrantTypes("authorization_code")
        .accessTokenValiditySeconds(120)
        .scopes("read_profile", "read_contacts");
    }
}
```

6. application.properties

spring.application.name=**authorization-code-type-server**

server.port=12345

security.user.name=srinivas

security.user.password=srinivas

7. pom.xml

<project ...>

<modelVersion>4.0.0</modelVersion>

<groupId>**com.coursecube.oauth2**</groupId>

<artifactId>**authorization-code-type-server**</artifactId>

<version>1.0</version>

<packaging>jar</packaging>

<name>**authorization-code-type-server**</name>

<url>https://www.coursecube.com</url>

<description>**This is authorization-code-type-server developed by Srinivas Dande**</description>

<properties>

<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>

<java.version>1.8</java.version>

</properties>



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Implicit Grant Type

- The Implicit Grant Type is a way for a single-page JavaScript app to get an access token without an intermediate code exchange step.
- It was originally created for use by JavaScript apps (which don't have a way to safely store secrets) but is only recommended in specific situations.
- At a high level, the flow has the following steps:
 - 1) The application opens a browser to send the user to the OAuth server
 - 2) The user sees the authorization prompt and approves the app's request
 - 3) The user is redirected back to the application with an access token in the URL fragment
- In general, there are extremely less circumstances in which it makes sense to use the Implicit grant type.
- The Implicit grant type was created for JavaScript apps while trying to also be easier to use than the Authorization Code grant.
- The main downside of the Implicit grant type is that the access token is returned in the URL directly, rather than being returned via a trusted back channel like in the Authorization Code flow. The access token itself will be logged in the browser's history, so most servers issue short-lived access tokens to mitigate the risk of the access token being leaked

Working Steps:

- 1) Copy authorization-code-type-server as implicit-type-server
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Update **MyAuthorizationServer.java**



Running Steps:

Step1: Send the Request to Authorizarion Server with the following URL to Get the User's Permission

A) Eendpoint - /oauth/authorize

B) Query Parameters

- client_id,
- redirect_uri,
- response_type,
- scope

`http://localhost:12345/oauth/authorize?client_id=jlcapp&redirect_uri=http://localhost:5000/callback&response_type=token&scope=read_profile&state=hello`

- Once the Approved by Resource Owner , you will be redirceted to redirect-uri given with Access token as follows.

`http://localhost:5000/callback#access_token=ef5c8cd7-3833-4451-8176-ce355dfc3e80&token_type=bearer&state=hello&expires_in=43199`

Step 2: Now access the BookPrice by providing the retrieved access

You can now Invoke your Resources or REST API with the Access Token received above.

`curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization: Bearer ef5c8cd7-3833-4451-8176-ce355dfc3e80"`

implicit-type-server – Files Required

1. MyApplication.java	Same as Above
2. BookPriceInfo.java	Same as Above
3. BookPriceController.java	Same as Above
4. MyResourceServer.java	Same as Above
5. MyAuthorizationServer.java	Updated in Implicit Type
6. application.properties	Updated in Implicit Type
7. pom.xml	Updated in Implicit Type



5. MyAuthorizationServer.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.config.annotation.configurers.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Configuration
@EnableAuthorizationServer
public class MyAuthorizationServer extends AuthorizationServerConfigurerAdapter {
    @Override
    public void configure(ClientDetailsServiceConfigurer clients) throws Exception {

        clients.inMemory()
            .withClient("jlcapp")
            .secret("jlc123")
            .redirectUri("http://localhost:5000/callback")
            .authorizedGrantTypes("implicit")
            // .accessTokenValiditySeconds(120)
            .scopes("read_profile", "read_contacts");
    }
}
```

6. application.properties

```
spring.application.name=implicit-type-server

server.port=12345

security.user.name=srinivas
security.user.password=srinivas
```

7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>implicit-type-server</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
```



```
<name>implicit-type-server</name>
<url>https://www.coursecube.com</url>
<description>This is implicit-type-server developed by Srinivas Dande</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
<java.version>1.8</java.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Resource Owner Password Credentials Grant Type

- The OAuth 2.0 Password Grant Type is a way to get an access token given a username and password. It's typically used only by a service's own mobile apps or JavaScript Apps and is not usually made available to third party developers.
- The Password grant is one of the simplest OAuth grants and involves only one step: Makes a POST request to the server to exchange the password for an access token.

Working Steps:

- 5) Copy authorization-code-type-server as **ro-password-type-server**
- 6) Update **pom.xml**
- 7) Update **application.properties**
- 8) Update **MyAuthorizationServer.java**

Running Steps:

Step1: Send POST Request to Authorization Server to get the Access Token

A) Endpoint - /oauth/token

B) Query Parameters

- ClientID
- Client Scret
- grant_type=password
- RO username
- RO password
- scope

```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H
"content-type: application/x-www-form-urlencoded" -d
"grant_type=password&username=srinivas&password=srinivas&scope=read
_profile"
```

Above POST request will send the Access Token in the Response as follows

```
{"access_token":"ed86b8be-cc1a-438e-8b77-7f0272f1743",
"token_type":"bearer",
"expires_in":299,
"scope":"read_profile"}
```



Step 2: Now access the BookPrice by providing the retrieved access

You can now invoke your Resources or REST API with the Access Token received above.

curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization: Bearer ed86b8be-cc1a-438e-8b77-57f0272f1743"

ro-password-type-server – Files Required

8. MyApplication.java	Same as Above
9. BookPriceInfo.java	Same as Above
10. BookPriceController.java	Same as Above
11. MyResourceServer.java	Same as Above
12. MyAuthorizationServer.java	Updated in password Type
13. application.properties	Updated in password Type
14. pom.xml	Updated in password Type

5. MyAuthorizationServer.java

```
package com.coursecube.oauth2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.oauth2.config.annotation.configurers.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
import org.springframework.security.oauth2.config.annotation.web.configurers.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Configuration
@EnableAuthorizationServer
public class MyAuthorizationServer extends AuthorizationServerConfigurerAdapter {

    @Autowired
    private AuthenticationManager authenticationManager;
```



```
@Override
public void configure(AuthorizationServerEndpointsConfigurer endpoints) throws Exception
{
    endpoints.authenticationManager(authenticationManager);
}

@Override
public void configure(ClientDetailsServiceConfigurer clients) throws Exception {

    clients.inMemory()
        .withClient("jlcapp")
        .secret("jlc123")
        .authorizedGrantTypes("password")
        .accessTokenValiditySeconds(300)
        .scopes("read_profile", "read_contacts");

}
}
```

6. application.properties

```
spring.application.name=ro-password-type-server

server.port=12345

security.user.name=srinivas
security.user.password=srinivas
```

7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>ro-password-type-server</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>ro-password-type-server</name>
<url>https://www.coursecube.com</url>
<description>This is ro-password-type-server by Srinivas Dande</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
```



```
<java.version>1.8</java.version>
</properties>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Client Credentials Grant Type

- The Client Credentials flow is intended for server-side (confidential) client applications with no end user, which normally describes machine-to-machine communication.
- Your client application needs to have its client ID and secret stored in a secure manner.

Working Steps:

- 1) Copy authorization-code-type-server as **client-credentials-type-server**
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Update **MyAuthorizationServer.java**

Running Steps:

Step1: Send POST Request to Authorization Server to get the Access Token

A) Endpoint - /oauth/token

B) Query Parameters

- ClientID
- Client Scret
- grant_type=client_credentials
- scope

```
curl -X POST "http://localhost:12345/oauth/token" --user jlcapp:jlc123 -d "grant_type=client_credentials&scope=read_profile"
```

Above POST request will send the Access Token in the Response as follows

```
{"access_token":"126f398a-97da-423f-9464-68ab6f13cbfb",  
  "token_type":"bearer",  
  "expires_in":299,  
  "scope":"read_profile"}
```

Step 2: Now access the BookPrice by providing the retrieved access

You can now invoke your Resources or REST API with the Access Token received above.

```
curl http://localhost:12345/myapi/bookPrice/105 -H "Authorization: Bearer 126f398a-97da-423f-9464-68ab6f13cbfb"
```

client-credentials-type-server – Files Required

1. MyApplication.java	Same as Above
2. BookPriceInfo.java	Same as Above
3. BookPriceController.java	Same as Above
4. MyResourceServer.java	Same as Above
5. MyAuthorizationServer.java	Updated in client-credentials Type
6. application.properties	Updated in client-credentials Type
7. pom.xml	Updated in client-credentials Type

5. MyAuthorizationServer.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.config.annotation.configurers.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */

@Configuration
@EnableAuthorizationServer
public class MyAuthorizationServer extends AuthorizationServerConfigurerAdapter {

    @Override
    public void configure(ClientDetailsServiceConfigurer clients) throws Exception {

        clients.inMemory()
            .withClient("jlcapp")
            .secret("jlc123")
            .authorizedGrantTypes("client_credentials")
            .accessTokenValiditySeconds(300)
            .scopes("read_profile", "read_contacts");

    }

}
```



6. application.properties

spring.application.name=client-credentials-type-server

server.port=12345

7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>client-credentials-type-server</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>client-credentials-type-server</name>
<url>https://www.coursecube.com</url>
<description>This is client-credentials-type-server by Srinivas Dande</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
<java.version>1.8</java.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```